第六部分 · 代表性智能体记忆系统

MemGPT / Letta 虚拟上下文管理与分层长期记忆

从操作系统内存类比，到可编辑Memory Blocks，再到2026年的Git支持MemFS记忆文件系统。

SAVE → EVICT → RECALL → REASON → ACT

> 不是让窗口无限变大，
> 而是决定“什么值得现在看见，
> 什么应该留在外部，什么时候再找回来”。

零基础理论课 · 31张机制图 · 12道理论题与完整答案
v1.0 | 2026-10-19 | 仅理论，无实验与编程

第一部分｜本讲知识地图

先画出一个会“管理自己上下文”的智能体

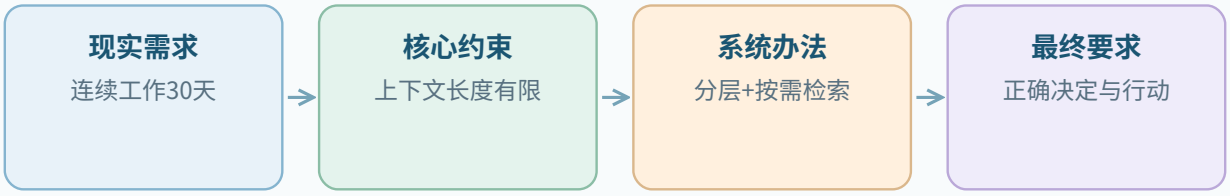


图1：从长期任务需求到按需装入：本讲的核心因果链

前27讲从记忆功能走到真实研究。这一讲只深入一个系统家族：**2023年提出的 MemGPT 思想，以及由其研究团队发展而来的 Letta。**目的是搞清楚：有限输入窗口下，智能体如何自己组织“本轮能看到什么”，如何把旧信息保存到外面，又怎样在需要时检索回来。

- 先分清**模型本身**、**上下文管理器**和**持久化存储**，再学习三者之间的信息流动。
- 读懂论文中 Main Context、Working Context、FIFO Queue、Recall Storage、Archival Storage、Queue Manager、Function Executor 的具体职责。
- 从内存压力触发到清理对话、递归摘要、分页检索、工具链，再推演一次完整的科研任务。
- 区分 Letta 的经典 Memory Blocks，以及2026年 Letta Code 的 Git 支持的 MemFS；它们都是工程实践，但不是同一个时期的同一个实现。
- 从原论文实验结果和失败案例判断方法为什么有效，以及在哪些条件下可能没有价值。

> **要点：**学习目标不是记住一大串产品名称，而是能够**独立推演“什么在窗口里，什么不在，如何移动，何时会出错”**。

第一部分｜贯穿全讲的例子

先认识一个持续30天的科研助手



图2：同一项目中的历史建议、正式决定、故障与新证据必须区分

以下是为教学**虚构**的研究助手。P、Q为两个不同项目：它必须记得研究者的决定、历史工具结果、旧建议，以及一次性许可是否已经使用。第30天的问题是：“现在能否恢复P项目的收费实验？”

正确答案不能由“语义最相似的记忆”决定，而应以当前项目状态和权威决定为准。如果第4天的建议被永久固定在核心记忆、或者第12天的暂停被淘汰，对话助手就会持续犯错。

- D01：研究者宣布P项目只进行离线研究；此为当前有效目标。
- D04：另一智能体建议给P项目发起收费调用；它只是建议。
- D09：工具返回HTTP 504；能证明请求失败返回，但不足以确定下游是否执行。
- D12：研究者再次暂停P的新增付费执行。
- D18：附加网关证据支持了D09事件的后续诊断。
- D26：仅给Q项目一次额度受限的操作许可，不能转移给P。
- D30：智能体必须回答P当前是否获准，并给出D12、D26的来源。

第二部分｜问题与系统灵感

为什么有很长上下文，仍然需要 MemGPT?

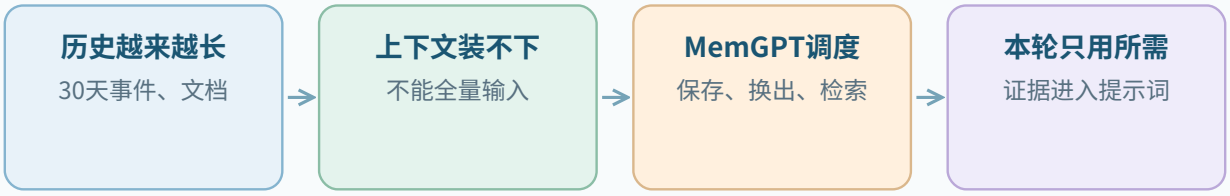


图3：虚拟上下文的因果逻辑：外部可存更多，本轮仍须选择

大模型生成下一步Token时，主要基于本次请求提供的Token；窗口之外的历史不会自动进入模型的注意力计算。聊天服务可以保存过去的字节，却必须决定把哪些信息**再次装入请求**。

固定回放全部历史的代价随对话增长；只保留滚动摘要又可能丢失一个否定词、一个原始错误码或一项生效时间。MemGPT将问题改写为：**模型能否参与决定哪些内容应常驻、哪些内容留在外部、何时查询回来？** [R1]

- **固定窗口：**模型当前直接利用的Token数量仍有限。
- **外部上下文：**历史消息、文件、归档可以大量保存，但需要检索才进入模型。
- **控制层：**队列管理器管理溢出，函数执行器真正落实模型提出的记忆操作。
- **系统级收益：**在多轮对话或长文问答中灵活分配上下文，而不是更改Transformer权重。

> **要点：**本质不是“把模型物理上下文窗口无限扩大”，而是创造**可多次检索、逐步处理的虚拟上下文体验**。

第二部分｜与操作系统的类比

虚拟内存给了什么启发，类比到哪里应当停止？

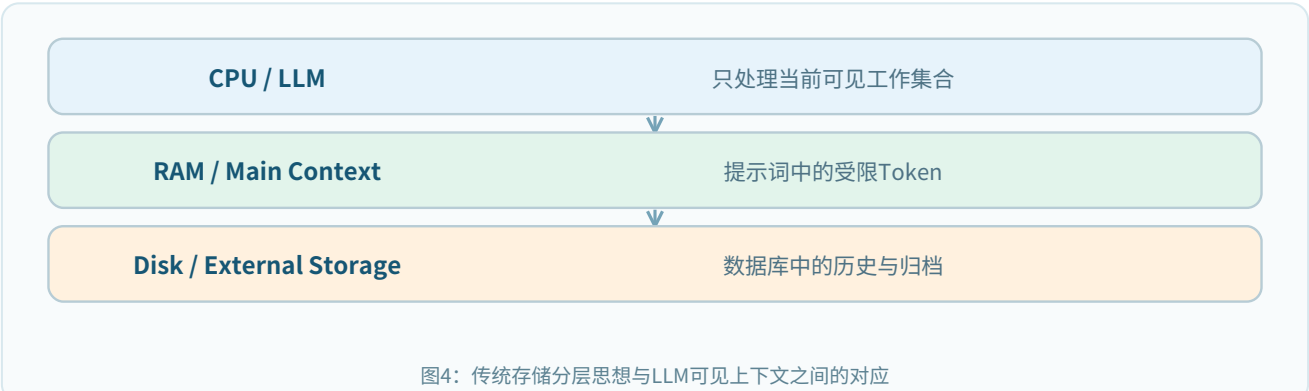


图4：传统存储分层思想与LLM可见上下文之间的对应

传统操作系统的虚拟内存给程序一个较大的连续地址空间，在物理RAM不足时通过操作系统机制管理内存页。MemGPT借用“快层有限、慢层较大、需要按需搬运”的思想。这里的快层是**本次输入Token**，慢层是外部存储。[R1]

计算机系统中的概念	MemGPT中的对应物	必须保留的区别
物理内存 RAM	Main Context：本轮输入Token	Token并不等于真实RAM页
磁盘/二级存储	Recall / Archival Store	保存不保证模型能正确找回
换入/换出	工具检索、记忆编辑与队列淘汰	常包含自然语言理解与生成
缺页与内存压力	提示缺信息或触发容量警告	不是CPU/MMU的硬件缺页中断
内存地址与页表	来源ID、记录ID、检索条件	没有操作系统式精确字节寻址保证

要点：重要边界：MemGPT的“分页”是系统架构类比，**不保证精确页换入、原字节无损恢复，也不保证LLM每次调用都正确**。

第三部分｜原始论文的完整架构

MemGPT原论文：一层Main Context、两种外部存储

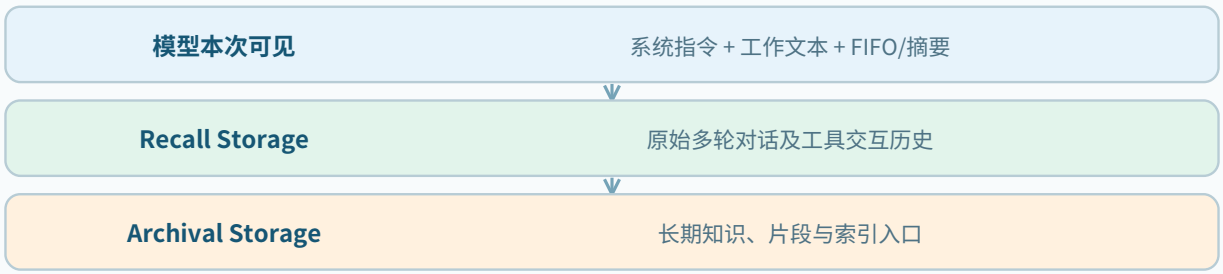


图5：原论文中主上下文与两类外部存储的功能划分

MemGPT把上下文组织为**Main Context**和**External Context**两大类。Main Context拆为系统指令、Working Context、FIFO Queue。External Context则包含Recall Storage与Archival Storage。两者的访问方式、语义与典型内容不同。[R1]

- 系统指令定义可用的记忆工具和控制流程，在论文设计中通常只读。
- Working Context是可由专用函数改写的短文本区域，用于身份、重要人物与持久目标。
- FIFO Queue存放近期多轮消息、工具调用和结果，并包含历史摘要。
- Recall Storage是可搜索的对话记录，历史消息被挤出窗口后仍可在此查找。
- Archival Storage是可写、可搜索的长期归档，用于文件片段、知识和经历等。

> **要点：**只有Main Context会被直接作为本次LLM输入；外部数据库不因“已经连接”就自动出现在模型的注意力中。

第三部分 | Main Context内部

本轮输入包含三个区域：系统、工作、先进先出消息



图6：系统、工作区、近期消息和输出预留共同争用有限容量

Main Context不是一个简单的“全部聊天记录字符串”。在论文设计中，它被划分成三段：**System Instructions**指导运行规则；**Working Context**保留有用的、可编辑的重要事实；**FIFO Queue**维持近期互动及历史摘要。

第30天科研助手能读到的内容例如：系统指令规定“事实与建议需区分”；工作区记录“当前任务P，D12暂停收费”；FIFO末尾是D30用户提问和刚查到的文件回执。即使D04旧建议出现在FIFO，也不得高于已核实的D12决定。

- System Instructions：主要负责工具、流程和身份/策略，不应被工具输出擅自改写。
- Working Context：可自编辑，但空间有限，写错可能对后续每轮产生持续影响。
- FIFO：越新的消息通常越靠近队尾，早期消息可能被清理和摘要替代。
- Recursive Summary：摘要保留概况，却可能丢失一个关键的否定、精确时间或来源。

> **要点：**Working Context中的“常驻”是指在相应交互设计下经常被拼进提示词；它不是模型参数，也不是永远不会被改写的绝对记忆。

第三部分｜Working Context的价值

为什么要把重要内容放进可编辑的工作记忆?



图7：不同信息保留层次：高频有效状态与长历史明细

如果P项目当前有效的费用限制每轮都要检索，系统会多一次查询、多几段上下文，也可能遇到搜索遗漏。因此，MemGPT允许智能体修改Working Context，持续保留少量高复用的信息。代价是**一条写错的“常驻事实”会反复影响未来回答**。

例如第12天项目暂停可以被写成“P：除新明确授权外禁止新增付费；来源D12”。如果记成“所有项目不准付费”，第26天的Q项目例外就会被错判；如果仅记成“曾经暂停”，第30天又可能误判P已经恢复。

- **应优先保留：**当前任务目标、用户明确而持续有效的偏好、关键限制和索引入口。
- **不应硬塞：**冗长原始日志、未经证实的AI推断、过时一次性计划。
- **修改条件：**新信息必须能标出来源、版本和适用范围；冲突时不要静默覆盖历史。
- **检索回源：**涉及权限、预算、诊断、合同等关键问题时，仍需查权威来源。

论文证据与推断：来源于原论文的是“可由模型经函数编辑短期工作文本”；字段、审批和来源指针要求是本课程的可靠性设计建议，不是原始MemGPT已提供的强制许可系统。

第三部分 | Recall与Archival

Recall Storage与Archival Storage究竟怎么不同?

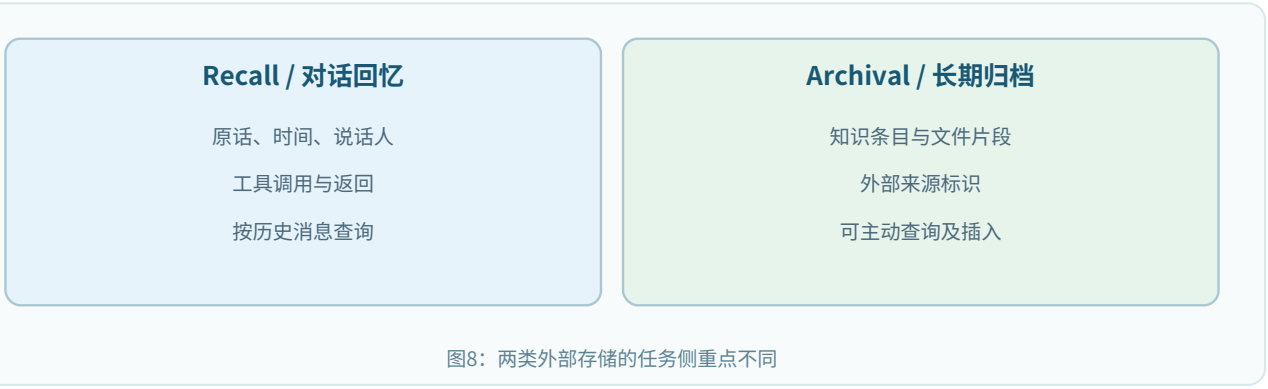


图8：两类外部存储的任务侧重点不同

Recall Storage更接近“我之前究竟说过什么”：包含历史对话和工具交互消息。Archival Storage更接近“我长期存有哪些知识材料”：可保存文档、片段、记忆条目或其他可搜索数据。[R1]

比较维度	Recall Storage	Archival Storage
主要对象	历史交互、消息、工具输入输出	文档、知识片段、主动保留的长期信息
典型检索	按时间、文本、消息记录搜索	语义搜索、标识符或索引检索
保存粒度	更重视原话与事件序列	可以按段落、事件或资料单元组织
是否可直接被模型看见	不能，检索后必须回填	不能，检索后必须回填
核心风险	定位错会话/时间，误读旧消息	相似度高但权威性或时效性不足

第30天若要确认第12天究竟如何表述暂停，Recall Storage可帮助定位原始发言；若要找论文中关于504故障的通用解释，Archival Storage可能更适合。但“相关”不意味着能覆盖本项目实时的权威工具回执。

> **要点：**现实系统可能把两种记录放入相同数据库引擎或用同一索引层；功能区别不意味着必须部署两套独立数据库。

第三部分 | Queue Manager

上下文快满时：谁负责预警、清理与保留？

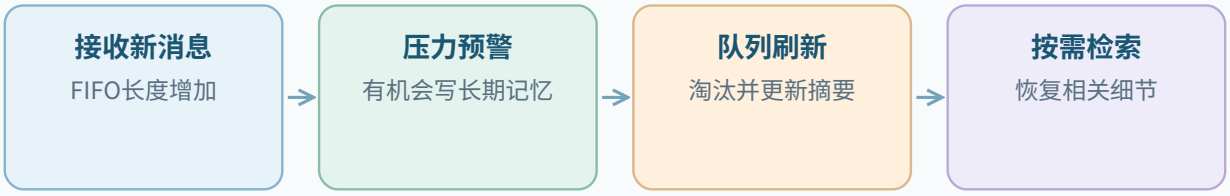


图9：容量管理的动态过程：预警与队列刷新不是同一事件

论文中的Queue Manager既负责把新消息加入FIFO，也负责把输入与输出写入回忆库。当主上下文逼近容量阈值，它插入一个“内存压力”警告，让模型有机会通过工具把重要内容迁移到Working Context或Archival Storage；到清理阈值再淘汰部分较早消息并递归摘要。[R1]

1. 接收新消息并追加到FIFO；同时把交互持久化到Recall Storage。
2. 检测当前Token使用量是否接近预警阈值（论文举例为70%）。
3. 提示模型：决定哪些重要事实需要额外保存。
4. 如果达到刷新阈值（论文举例接近100%），淘汰部分旧FIFO消息（例如淘汰一半）。
5. 用被淘汰消息和旧摘要生成新的递归摘要，保留在FIFO前部。
6. 继续处理本轮问题；必要时再主动搜索Recall/Archival取回遗漏细节。

> **论文证据与推断：**70%、100%和50%为原论文介绍的阈值/策略示例，不是每个模型、每个Letta版本的固定常数。

第三部分｜手算上下文预算

窗口不会魔法般变大：完整复算16,000 Token

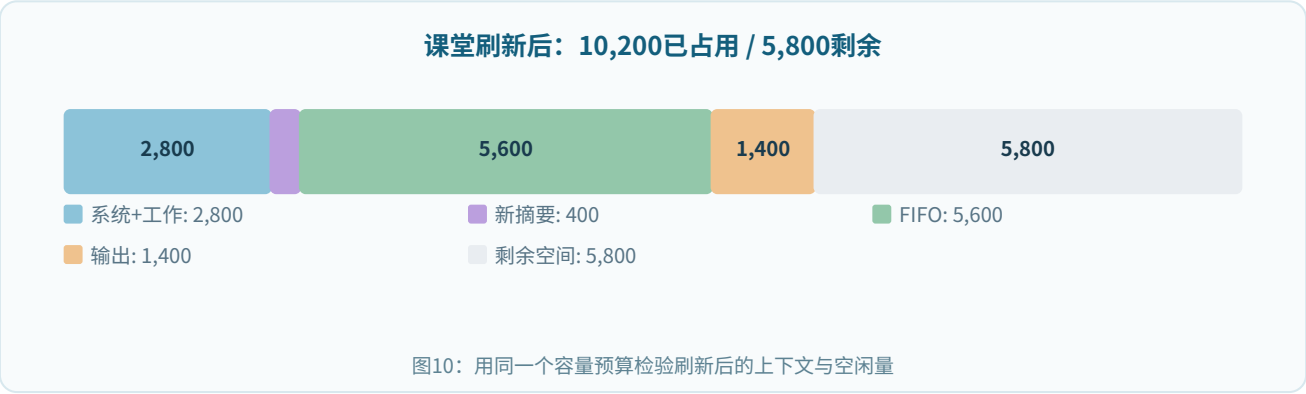


图10：用同一个容量预算检验刷新后的上下文与空闲量

设课堂虚构窗口上限为16,000 Token，原有系统指令1,500、Working Context 1,300、递归摘要700、近期消息9,600、输出预留1,400。总计14,500 Token，剩1,500。这时用户又发来2,000 Token信息，候选总量达到16,500，超过上限500。

假设Queue Manager在满足系统策略后清理6,000 Token的较早消息，用新400 Token摘要替换原700 Token摘要：新的工作量=1,500+1,300+400+(9,600−6,000)+2,000+1,400=10,200 Token，还余5,800。这里假设淘汰的原始消息在Recall库中可检索，并不代表摘要能无损保留。

项目	刷新前+新信息	刷新后
系统指令+工作区	2,800	2,800
摘要	700	400
FIFO含新消息	11,600	5,600
输出预留	1,400	1,400
总量	**16,500（超500）**	**10,200（余5,800）**

> **要点：**如果第12天暂停事实只出现在被删除的消息中，而新摘要遗漏它，容量虽然节省了，事实忠实度却下降。这就是“Token变少”和“长期记忆更好”不能画等号。

第四部分 | 工具执行器

谁真正把模型的“记忆操作”落实?

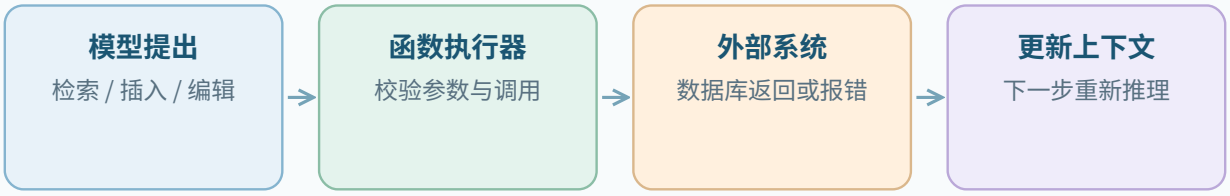


图11：MemGPT以函数调用作为主上下文与外存之间的桥梁

模型只能生成下一段输出，可以在该输出中提出“查询历史”“插入归档”“修改工作记忆”等工具调用。真正执行工具的是**Function Executor**及其外部存储接口，而不是模型“想了想”就已经写入了数据库。[R1]

1. 模型发现D30问题需要核对P项目当下费用状态。
2. 模型提出读取D12相关历史记录的函数调用。
3. 执行器检查函数名与参数是否合法，并实际访问Recall Storage。
4. 工具返回查找结果或错误；只有已返回的内容才能被加入Main Context。
5. 模型依据新证据进一步查询或生成回答。
6. 涉及真实收费操作时，控制器还应额外核验权限、预算与环境状态。

经典MemGPT实现也有用于修改Working Context或Archival Storage的工具，但不同版本函数命名与参数可能不同。**本课只讲职责与信息流，不把示意工具名当作当前API契约。**

> **要点：** “模型提出工具调用” ≠ “执行器已成功保存” ≠ “下游业务完成”。真正的事实证据来自工具回传和必要的环境核验。

第四部分｜控制流与Heartbeat

检索一个结果不够时，如何连续调用多次?

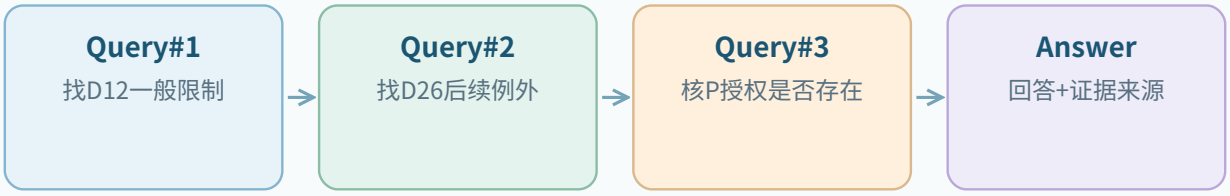


图12：多次工具调用改变的是可利用的信息集合，不是原始模型的上下文上限

有时第一条检索只回答“见过这项限制”，但还缺失它是否被后来替代。MemGPT允许模型通过函数调用的特殊控制参数请求**立即继续一次推理**，在工具返回后再检索下一段证据。论文中以`request_heartbeat=true`举例。[R1]

- 第一次：检索D12暂停决定，确认项目P被暂停。
- 第二次：检索D26是否存在后续例外，发现许可只限Q。
- 第三次：核验是否存在P的新授权记录；若无，不能把“不知道有没有新授权”伪装为肯定许可。
- 结束：形成带来源的回答，而不是无穷搜索和永远不回复用户。

论文将用户消息、系统压力警告、定时事件等统一视为可触发模型运行的事件。Heartbeat关注函数结果之后是否继续下一次模型推理，并不是心脏跳动意义的实时硬件机制，也不自动代表后台无限运行。

> **要点：**自主工具链需要次数和费用上限、停止条件和错误分类。MemGPT提出可重复检索的运行机制，但不能因此假定所有实现都有完整权限、审计和预算守卫。

第四部分 | 一个动态闭环

D30科研助手：从提问到回答，究竟发生了什么？



图13：长期科研助手从问题到记忆检索、再到行动控制的简化路径

把MemGPT的存储层、队列管理器、工具和LLM推理组合，便得到一个可追溯的循环。第30天用户询问“P项目能否恢复收费”时，系统首先从当前工作状态看到P仍被暂停，然后有针对性地取回D12原始决定和D26仅对Q有效的后续记录。

1. Main Context包含问题、少量当前状态与系统工具指令。
2. 模型识别“当前权限判断”是高风险关键事实，不依赖单纯语义相似排名。
3. 召回D12源记录与D26范围条件；如状态服务有权威当前值，则应先查它。
4. 执行器将结果带回，控制器对项目、时间、主体和额度作核验。
5. 模型生成“P没有新的收费许可，Q的例外不适用P”的结论。
6. 如果用户只是询问，不能擅自调用收费API；若提出收费行动，须有明确新增授权。

这是一套**融合原论文设计思想与课程可靠性约束的教学架构**，并非声称2023年的MemGPT论文原样实现了研究审批和费用限制。

> **论文证据与推断：** “查得到旧对话”是必要能力之一，却不是权限判断系统本身。关键项目状态应由独立于语言生成的权威规则核验。

第四部分 | 摘要故障

为什么递归摘要可能让Agent越记越错?

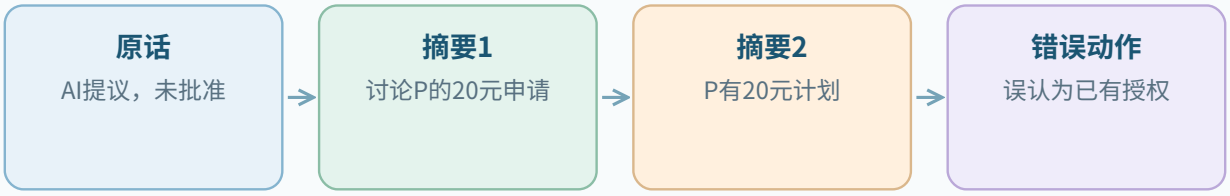


图14：摘要连续改写如何丢失来源、否定与时间边界

旧摘要可以缩短上下文，但摘要过程经常要进行语义取舍。假设D04消息是“AI建议为P申请20元，尚未批准”，第一轮摘要错误地保留成“讨论过P申请20元”；第二轮摘要又压成“P有20元计划”。经过几次改写后，否定和来源信息都消失。

- 第一步损失：**说话人**丢失，建议与负责人决定被混同。
- 第二步损失：**否定/模态**丢失，从“未批准”变成“有计划”。
- 第三步损失：**时间**丢失，过时信息被误作当前状态。
- 第四步放大：模型根据错误工作记忆生成新的计划，并把计划再次写回。

可靠方案应保留事件原文、时间与出处索引，用当前状态数据库核验关键事实，并避免由递归摘要自我证明其真实性。即使摘要具有“来源链接”，也要检查来源能否支持具体主张。

> **要点：**可追溯性有助于发现错误，但链接存在不等于引文本身足以证明结论。

第五部分 | 历史沿革

MemGPT（论文）与Letta（框架）怎样连接?



图15：从研究概念到产品工程实践的时间脉络

MemGPT首先是2023年的一个研究项目及其提出的系统思想。随着开源实现发展，团队于2024年将更广泛的状态化Agent框架品牌调整为**Letta**。官方也明确说明“MemGPT”可能指论文方法、代理类型或早期软件，因此混用名称容易误解。[R2]

时间层次	名称/对象	本讲应怎样理解
2023 / 2024论文	MemGPT原始研究	主上下文—Recall—Archival及自主工具控制
2024及其后	Letta框架	可持久化的状态化Agent、记忆块与工具接口
2025部分API	Memory Blocks	常驻上下文的命名记忆单元，可单独更新
2026 Letta Code	MemFS / Context Repository	把Agent记忆映射为Git支持的可编辑文件系统

这不是“产品所有版本功能完全相同”的时间线，也不是每个Letta Agent都同时使用全部产品功能。特别是Letta Code与API模式的文档在术语、存储管理、前端工作流上可能不同。

> **论文证据与推断：**后续页面将分别介绍Classic MemGPT、Letta Memory Blocks、Letta Code MemFS。引用时应标明**文档年份、产品表面与实现类型**。

第五部分 | Memory Blocks

Letta Memory Block是什么？它为什么“常驻”？



图16：Memory Blocks适合高频短信息，外部资料更适合长历史与证据

在Letta经典/SDK语义中，一个Memory Block是给Agent的上下文保留的一段**具名文本区域**，具有标签、内容、容量上限和说明。它可以持久保存到Agent状态，并在后续交互中再次放进系统提示中，因此不需要每次都先做语义检索。[R3][R4]

例如把“研究者偏好零成本先验证”放在`user_preferences`块；把“本轮需要确定P项目是否获得新授权”放在当前线程状态；而长达300行的D09代理日志则不应常驻记忆块。

字段（教学表达）	含义	容易犯的错
label: project_policy	说明用途的名字	命名为policy不能证明文本真的获批
value: “P暂停收费”	进入模型上下文的文本	内容可能过期或范围错误
limit: 最大字符数	约束可占上下文容量	并非保证所有模型统一的Token配额
description: 如何使用	告诉Agent该字段代表什么	说明文字本身不能改变真实工具权限
read-only / editable	哪些角色可修改	是否可改取决于当前产品权限与配置

> **要点：** “Memory Block是持久保存的”与“模型参数永久学会”不同：通常仍是将块内容以提示词形式注入，而不是继续训练模型权重。

第五部分 | 块编辑与状态

一条Block被修改后，究竟改变了什么？

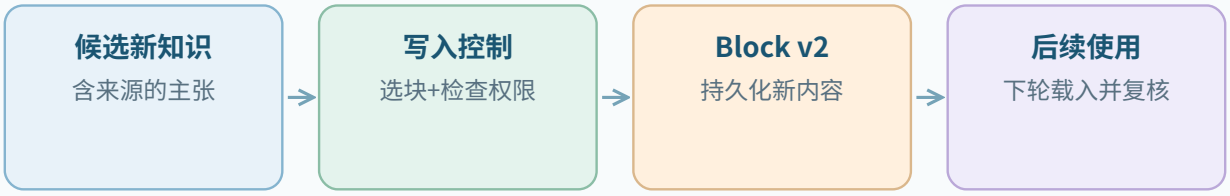


图17：Memory Block的更新是存储层状态变更，必须校验而非只信模型自述

Letta文档通过Agent/Block API提供独立读取、更新与关联操作。核心机制是改变保存的块内容或其挂载状态，后续推理再使用新值。模型输出“我已经记住了”并不等于接口真正保存；只有完成操作并核对持久状态才可确认更新。[R3][R5]

1. 在事件记录中识别一个可能值得长期保存的项目状态。
2. 确认来源主体和当前有效时间，决定应写哪个命名块。
3. 通过受控工具提交块更新（教学概念：旧版本V1→新版本V2）。
4. 校验API是否成功并读取块确认；必要时保存历史来源指针。
5. 下一轮模型获得V2块文本，但仍需对关键授权核实外部权威。

如果两个Agent同时改同一块，可能有丢失更新与互相覆盖的问题。版本校验、变更历史和分工可缓解，但不能靠“块本身可编辑”自动保证并发一致性。

> **要点：**在可编辑的Memory Blocks中，“模型能自主改写”是能力描述；“是否有权改写某个项目决定”是另外的权限问题。

第五部分 | 持久状态与摘要

Memory Blocks、会话历史、摘要和检查点不能混用



图18：模型提示词、记忆/历史、真实环境状态不属于同一层

同一个Agent有多种不同状态：当前提示词、已保存的记忆块、历史对话、工具执行记录、经过压缩的摘要，以及运行中的步骤/线程状态。它们都与“过去的信息”有关，却不解决同一个问题。

内容	主要用途	若混淆会怎样
Memory Block	长期可复用的短文本	把过时决定固化为常驻事实
会话/消息历史	保留某次真实对话与工具结果	仅有原文不能保证每轮可见
摘要/Compaction	压低上下文Token消耗	摘要漏掉否定词而改变意思
线程/运行状态	恢复正在执行的工作流	把“已计划”误认“已执行”
持久任务授权	核验哪个行动真正可执行	由AI的自然语言记忆冒充审批

Letta API文档中可以看到不同的消息压缩模式与配置，它们是上下文管理方案的具体接口，不代表历史消息删除等同于数据彻底删除。[R5]

> **要点：**最稳妥的推演：**谁是权威来源？什么被持久化？哪部分被载入？修改可否追踪？**四个问题分别回答。

第六部分 | 2026的新形态

MemFS：用Git支持的记忆文件系统

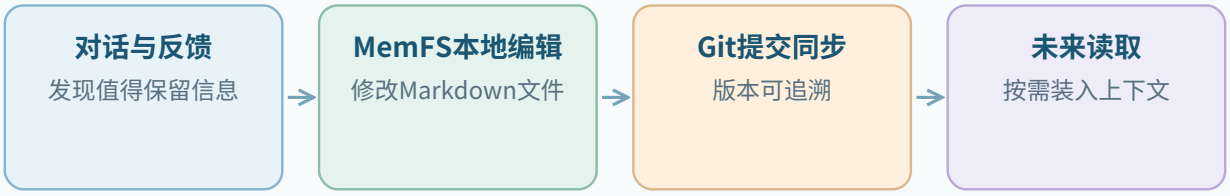


图19：MemFS中由文件编辑、提交到再次使用的状态流

Letta于2026年介绍了Context Repositories，Letta Code相关文档使用**MemFS**表示一种由Git管理的记忆文件系统。Agent可以把可保留知识写成文件，利用普通文件工具读取、编辑；提交和同步之后，让后续会话使用这些内容。[R6][R7]

相比早期依赖专用函数编辑Memory Blocks的方式，MemFS更便于按目录组织知识、查看修改历史、比较差异和协作合并。**它解决的是可管理性、版本化、渐进加载等问题，而不是让LLM在一次推理中无限读取磁盘。**

- 文件可以按项目、人物、流程拆分；短索引告诉Agent从哪里进一步加载。
- Git提交保留文件变化记录，便于审计和回退，但Git历史本身不是事实核验机制。
- 多个会话或子Agent的修改可能冲突，仍需合并策略与权威来源约束。
- 记忆写入文件本身并不自动进入下轮上下文，取决于系统何时注入或读取。

> **论文证据与推断：**MemFS是Letta当前文档描述的记忆组织形式；不要把2026年Git文件系统反推成2023年MemGPT论文原始实现。

第六部分 | MemFS的组织

记忆文件并非越大越好：索引与渐进披露

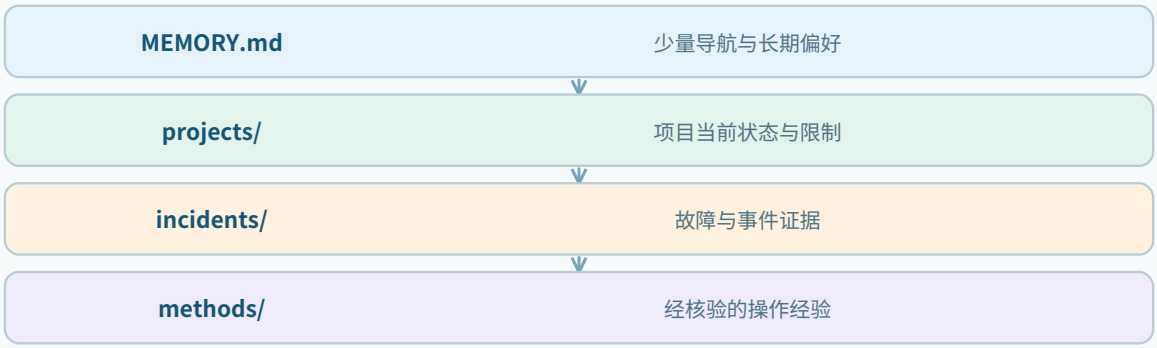


图20：目录索引与主题文件分离，有利于按任务逐层披露

在MemFS中，某些短信息适合留在每轮常用区域，另一些长资料按路径检索或按需展开。可以把它想成“先看到书的目录，再按问题打开一章”，而不是“每天把整座图书馆搬进教室”。[R7]

教学示例：顶层`MEMORY.md`写研究项目索引；`projects/P.md`保存P项目现行规则和来源指针；`incidents/D09.md`保存504事件与后来归因的历史；`methods/recovery.md`保存经验流程。文件名只是组织方式，仍需区分规范来源与摘要。

- **索引尽量短：**放“应到哪里找什么”，而不是塞入全部日志。
- **正文保持可追溯：**重要事实保留事件ID、适用时间和证据出处。
- **分块不要乱：**不同文件分担不同范围，减少互相覆盖和反复改写。
- **检索仍然必要：**信息存于Git仓库并不自动在本次Prompt中。

> **要点：**这套教学目录示例并非Letta强制目录布局；具体文件形式与MemFS接口随版本变化。

第六部分｜版本化的意义

Git历史能回答“谁改了什么”，不保证“改得对”

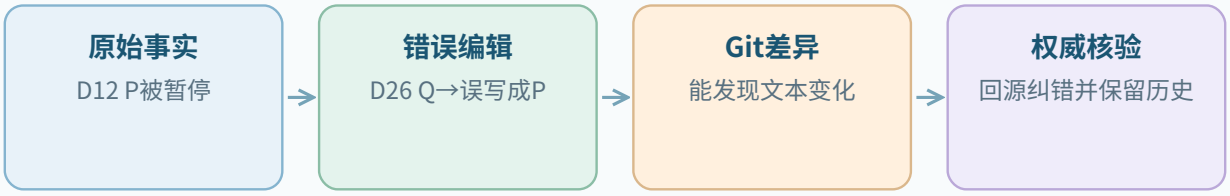


图21：版本化让错误可见，但纠错仍依赖来源与语义核验

假设D12的P项目暂停写在`projects/P.md`。D26有人错误把Q的一次许可合并成“P也获准”。Git可以显示两次文本之间的差异，也可能允许回退到D12版本；但它不会自动判断Q与P是否同一个项目，也不会证明修改者具有授权权利。

问题	Git/文件系统通常能提供	必须额外判断
哪一行何时被改?	提交历史、diff	修改是否真的来自有权决定者?
新旧文本有什么区别?	对比版本	变化是否符合项目作用域/时间?
并发编辑冲突?	分支和合并	合并结果是否产生逻辑矛盾?
恢复旧版本?	checkout/revert概念	是否会误删后续有效决定?

因此，可靠智能体记忆最好同时保持“版本结构完整”和“证据链语义正确”。前者是工程完整性，后者涉及事实、时间、权限和任务目标；二者不可互相替代。

> **要点：**把Git和科研证据链同时维护，能够帮助定位错误的第一次引入点，但不能直接把某次提交当作科学上正确的真值。

第六部分 | Dreaming / Sleeptime

让记忆在空闲时整理：收益和边界

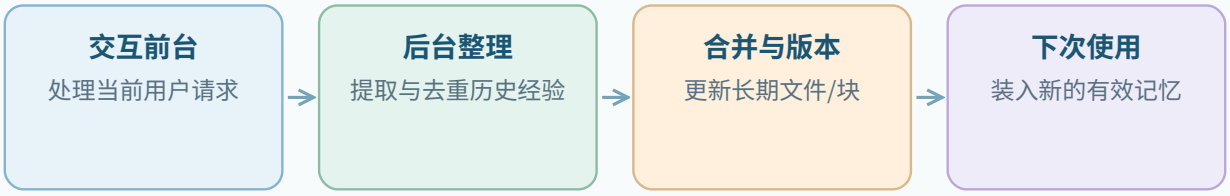


图22：把可延期的记忆整理移出前台，关键授权仍应同步生效

Letta的当前文档还介绍了在后台整理记忆的Dreaming/Sleeptime思路：在不打断当前任务的情况下，回顾近期对话，提取长期有用信息并更新记忆。这相当于把部分“记忆巩固”从用户等待的关键路径移走。[R8]

- **潜在收益：**减少每轮即时写入Token、批量合并重复经验、整理不同文件、保留变更记录。
- **潜在失败：**异步滞后导致下一次对话仍读旧事实；自动摘要可能误记推断。
- **控制要求：**关键权限变动不能等后台整理结束才生效；后台整理不应自行扩大收费调用范围。
- **成本要求：**后台步骤也消耗模型调用、计算与存储；不能当作“免费提高记忆质量”。

教学例子：研究者在D12宣布“P立即暂停付费”。执行层的权威状态应该立即改变；即使记忆整理任务到D13才把此事实写入归档，也不能允许D12晚间的付费动作通过。

> **要点：**本文讲的“后台整理”是一种系统机制。当前这份课程没有实际启动Letta或任何后台Agent，也没有授权长期无人值守服务。

第七部分 | 三代方案对照

原论文、Memory Blocks与MemFS不要混为一种架构



图23：技术思路连续，具体工程形态逐步变化

从科研阅读的角度看，MemGPT原论文研究了“模型控制上下文与外部记忆”的系统原理；Letta后续Memory Blocks强化了状态化Agent和具名内存；2026年的MemFS则把可编辑的长期上下文组织映射到Git支持的文件系统。它们在历史发展上有关联，但具体接口、控制粒度与失效机制不完全相同。

维度	MemGPT论文(2023)	Memory Blocks(后续SDK)	MemFS(2026 Letta Code)
关注	虚拟上下文调度	可持久命名上下文	文件/版本与渐进读取
常用表示	Working Context文本	标签+值+说明+大小限制	路径/Markdown/索引
外部访问	Recall/Archival工具	API与外部资料接口	文件工具+仓库同步
维护	函数+队列管理器	Block更新/关联	Git提交/差异/合并
主要风险	摘要损失、检索停止过早	常驻块过时、覆盖错误	文件冲突、过量注入、同步滞后

三种方案都不等于“某一种数据库自动拥有记忆能力”。关键在于谁触发写入与检索、什么进入上下文、如何检测错误、怎样组织来源与有效时间。

> **要点：**Letta持续迭代，当前页面的具体产品配置必须以相应版本官方文档为准；本课程只构建可迁移的概念模型。

第七部分｜原始实验与指标

论文真正验证了什么？先看多会话回忆任务

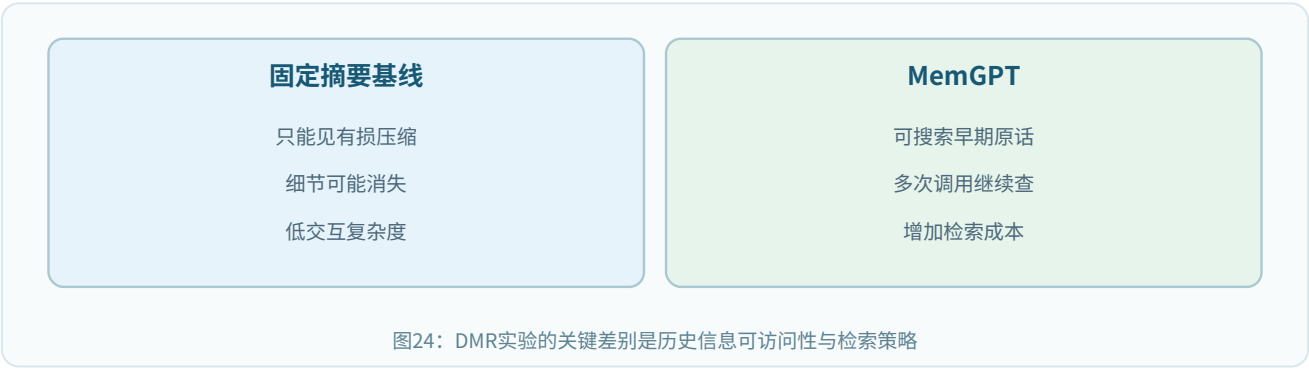


图24：DMR实验的关键差别是历史信息可访问性与检索策略

MemGPT作者在扩展的Multi-Session Chat设置中设计Deep Memory Retrieval（DMR）任务，让系统回答前几轮聊天中特定但不一定在摘要中的细节。对比方法使用历史对话的有损摘要，MemGPT则可通过查询取回更完整的聊天记录。[R1]

底座模型	固定上下文基线Accuracy	＋MemGPT Accuracy	提升幅度（百分点）
GPT-3.5 Turbo	38.7%	66.9%	+28.2
GPT-4	32.1%	92.5%	+60.4
GPT-4 Turbo	35.3%	93.4%	+58.1

这些数字来自论文表2。在该构造的多会话检索任务和特定模型/评测协议下，作者报告MemGPT比摘要基线更容易找回早期精确信息。它不等于在所有真实长期Agent任务中提升相同幅度。

- 不同系统能够看到的外部历史资料并不相同，因此测量的是系统级可获取信息与检索策略的合成效果。
- 作者使用生成的问答对与LLM评审；不能把评审准确率等同独立人类真值。
- 额外检索回合可能增加成本与延迟；该表不是质量—成本完整报告。

论文证据与推断：不要把“GPT-4+MemGPT的92.5%”泛化为“所有Agent记忆正确率92.5%”。必须保留底座模型、数据集、问题构造、基线输入及评分器版本。

第七部分 | 长文问答与多跳

长文档检索的优势在哪里，又为何仍会失败？

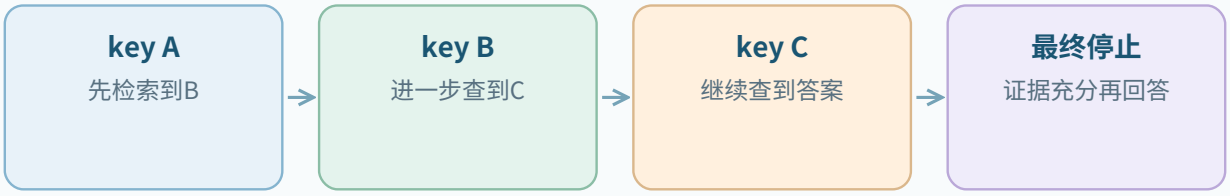


图25：嵌套KV多跳：模型要持续调用工具，不能只查第一步

论文还考察文档问答和嵌套键值检索。长文问答中，MemGPT可对外部文档反复检索并分页取回；嵌套KV则要求从第一个键找到下一个键，再持续查找直至最终答案。原论文使用由140个键值对构成的任务，并考察0—4级嵌套。[R1]

作者报告，在其嵌套KV任务下，使用GPT-4的MemGPT能够完成更深层的连续检索；但其它底座模型表现并不一致，GPT-4 Turbo搭配MemGPT在该任务里甚至不如GPT-4搭配MemGPT。这提示多步工具能力和停止策略会影响整体表现。

- 检索器可能把正确文档排得很后，Agent若提前停止分页仍会失败。
- 压缩或裁剪文档可能把答案所在片段删掉；完整外部存储不代表一定找到正确位置。
- 跨多次调用的上下文拼接会带来额外误读机会，复杂任务依赖模型正确使用函数。
- 在论文问答中能检索多轮，不自动代表具备真实研究授权和安全执行能力。

> **论文证据与推断：**原论文中的文档问答对照使用相同检索器，并且在部分设置下由LLM评估答案；它主要支持特定长期上下文访问论证，尚不能证明现实场景中最优成本或可靠性。

第八部分 | 可靠性与成本

一个好记忆系统还应回答：用了多少资源？

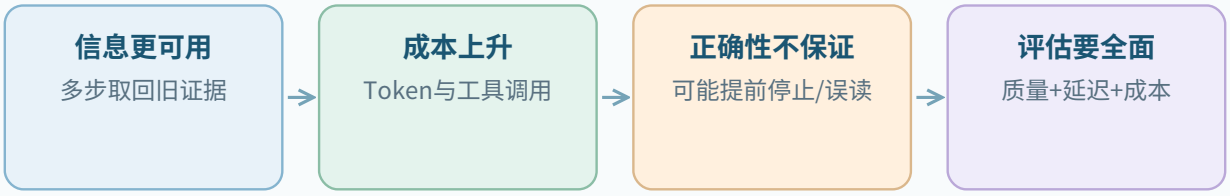


图26：虚拟上下文并非无成本的无限信息处理

MemGPT用上下文管理换取可访问的历史范围，但代价是更多检索和可能的多次模型推理。假设某课堂任务一次直接回答需要输入4000Token、输出500Token；引入外部记忆后先查询两次，每次额外输入800Token并输出120Token，随后最终回答输入4800Token/输出500Token。

两次查询与最终回答的总模型输入为800+800+4800=6400 Token，总输出为120+120+500=740 Token；相对一次直接回答多了2400输入和240输出Token。若按每千输入Token计费a元、输出Token计费b元，增量成本为2.4a+0.24b元。

方案	输入Token	输出Token	可获得的历史
单轮直接回答	4,000	500	仅本轮已装入的部分
两次查询+回答	6,400	740	两次查询逐步取回的证据
增量	+2,400	+240	检索不一定全部成功

真实系统还需统计向量检索/数据库耗时、网络往返、LLM调用延迟、写入更新成本、错误检索造成的额外成本，以及后台巩固的费用。不能只比较平均每轮Prompt长度。

> **要点：**上述Token与价格变量a、b仅为课堂代数演算，不代表Letta、MemGPT或任何具体模型的收费水平。

第八部分 | 失效机制

从“没记住”追溯到哪一层真正出了问题

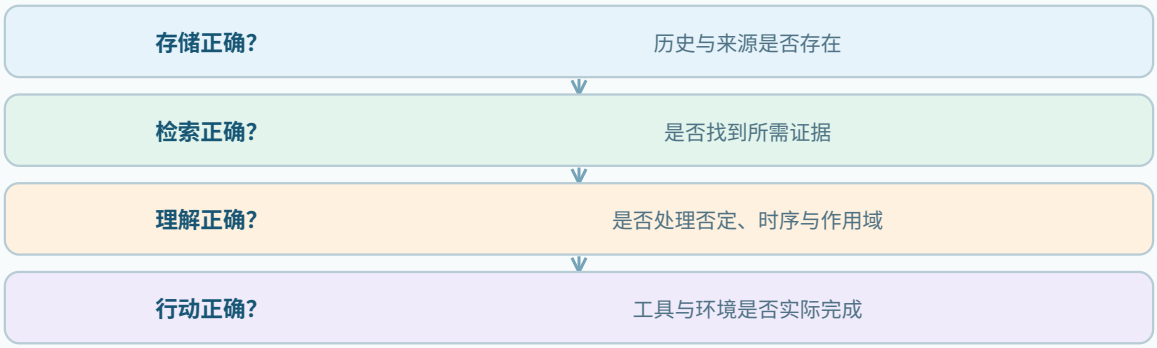


图27：分层诊断比统一标注“记忆失败”更有解释力

看到智能体答错长期历史问题，不应该立刻归因于“记忆系统太差”。要把错误拆到原始保存、写入/维护、分页检索、上下文组装、语义解释、控制执行这几个不同位置。

具体失效	直接症状	优先核查的层
D12决定未持久保存	Recall查询完全找不到	队列保存、数据库状态
D12仍在库中但未被检索	结果只出现D04建议	检索触发、分页停止
已检索却没进当前提示	工具返回被截断	组装与容量策略
已阅读但理解错主体	把Q例外当P许可	模型解释与时间/作用域
判断正确却执行错工具	收费请求被发出	执行器和权限控制
执行后状态不同步	日志成功但状态仍旧	工具回执、存储副本

评估中应保留每层可见输入、真实工具输出、版本、候选集与最终动作；LLM给出的长篇解释只是文本，不是其内部实际决策原因的独立证明。

> **要点：**针对高风险业务，必要的硬性权限验证应位于执行层，不能因核心记忆或最新摘要写着“已授权”就略去检查。

第八部分｜强简单基线与科研判断

使用MemGPT思想时，先比较简单策略是否已经够用

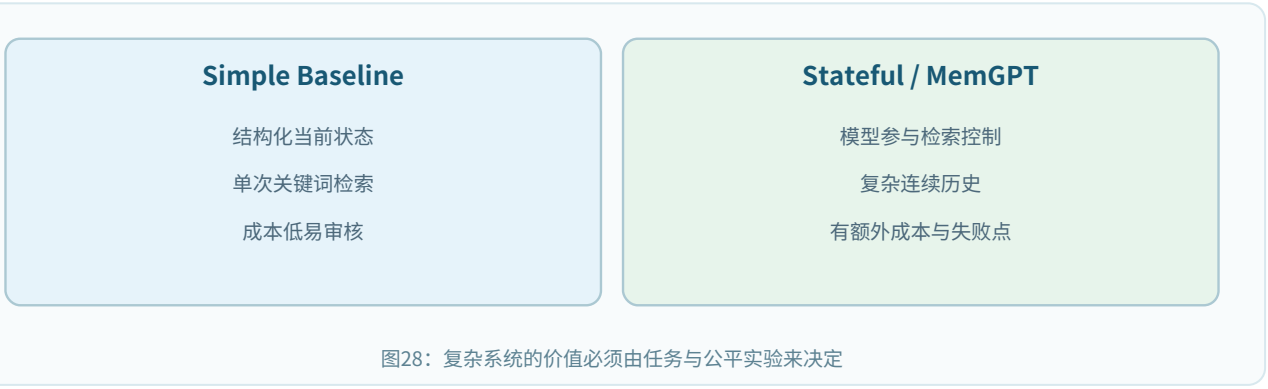


图28：复杂系统的价值必须由任务与公平实验来决定

在科研助手中，当前有效费用许可只是一条结构化状态。对这类问题，先读一行权威记录可能比启动三轮模型检索更直接可靠。对于跨多年开放式对话细节，MemGPT式按需回忆更有可能发挥价值。**架构应由任务需求决定，而不是越复杂越先进。**

任务	可能足够的简单基线	可能需要分层Agent的条件
当前P是否可收费	权威状态表+一次检索	需要说明跨会话决定演变过程
D09发生了什么	关键词/时间定位原始日志	证据散落多轮并需迭代查询
超长资料问答	强RAG一次或分批检索	答案需要跨多次有状态查询
长期陪伴对话	固定短摘要+检索	自适应记忆控制带来稳健收益

有科研价值的问题可包括：不同任务的检索深度何时有用？Memory Block错误如何长期传播？异步整理与当前状态更新如何避免竞争？何时一轮简单检索比自主反复搜索更好？这些是候选研究问题，不能直接宣称存在已证实的新机制。

> **论文证据与推断：**论文研究强调在可比条件下检验因果增量。先对齐模型、任务、历史可见性、Token预算、评价合同，再比较Simple、RAG、MemGPT式控制。

第九部分 | 12个最容易混淆的点

常见误区：知道表面名词不等于看懂运行机制

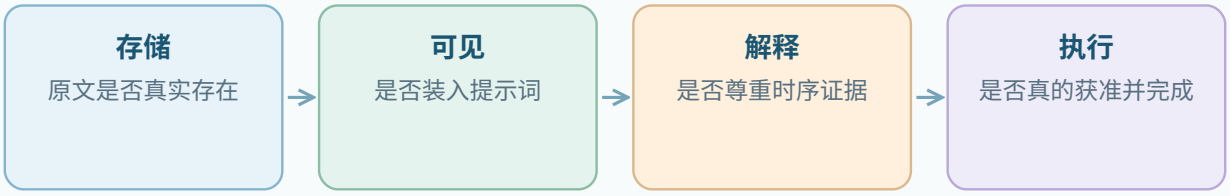


图29：贯穿本课程的四个不同责任层

- ① **“MemGPT=无限真实上下文”**：错误；每次LLM调用仍受固定窗口限制。
- ② **“外部信息存着模型就能看到”**：错误；需要检索并装入本次请求。
- ③ **“Working Context=KV Cache”**：错误；前者是可编辑语义文本，后者是推理缓存。
- ④ **“队列淘汰=删除对话原文”**：错误；论文中历史消息可保存在Recall。
- ⑤ **“递归摘要等于无损压缩”**：错误；否定、时间、来源都可能丢失。
- ⑥ **“MemGPT自己会管理一切”**：错误；工具调用可能失败，解析与检索需校验。
- ⑦ **“有Heartbeat=永不停止”**：错误；它是继续推理的控制信号，不是无限授权。
- ⑧ **“Memory Block就是外部向量数据库”**：错误；Block是命名、常驻上下文片段。
- ⑨ **“Letta所有版本和原论文完全一致”**：错误；产品接口与MemFS已经演进。
- ⑩ **“Git提交的记忆一定正确”**：错误；版本化不是事实或授权核验。
- ⑪ **“论文DMR指标可以直接预测真实科研助手成功率”**：错误；任务和基线不同。
- ⑫ **“复杂分层一定胜过简单查询”**：错误；质量、成本、权限与维护都要比较。

要点：这一页的目标是让概念回到**系统状态和运行过程**。每个误区都可反推到“谁保存、谁看见、谁执行、谁核验”四个问题。

第十部分 | 练习①

理论思考题（1—6）：先独立画图再看答案

> **要点：**练习1—6侧重架构、预算与运行流程；建议不看答案，先在一张纸上画出主上下文、外部存储和四种信息移动的箭头。

图30：None

1. 请用通俗语言解释MemGPT“虚拟上下文”的本质：外部存储为什么可以很大，但单次LLM上下文仍有限？举一个科研助手例子。
2. 原论文Main Context三部分和External Context两部分分别是什么？请画出信息流向，并标出模型能直接看见哪一部分。
3. Working Context、Recall Storage、Archival Storage分别更适合保存哪些信息？对D12暂停决定、D09原始504日志、外部论文片段分别作选择并解释原因。
4. Queue Manager在容量压力出现时通常会做什么？请明确预警、写入外存、FIFO淘汰与递归摘要的先后逻辑；摘要后是否能保证原细节仍在上下文?
5. 课堂窗口16,000 Token，系统+工作2800、旧摘要700、旧FIFO9600、预留输出1400；再来2000Token新消息。求超额值。若淘汰旧FIFO 6000并将摘要改为400，新的总用量、剩余是多少?
6. 函数执行器如何与LLM交互？“模型调用了检索工具”这句话为什么不足以证明结果已经保存或任务已经完成?

第十部分 | 练习②

理论思考题（7—12）：理解产品演进与科学证据

> **要点：**练习7—12侧重功能链、版本化、研究结果解释、资源成本和可靠行动。没有代码要求。

图31：None

1. MemGPT中的`request_heartbeat`要解决什么问题？以“D12暂停→D26例外”设计一个最多两次额外检索的有界过程，并写出停止条件。
2. Letta Memory Block与普通聊天历史、模型参数分别是什么关系？一个可持久化Block为何仍会占用上下文Token？
3. 2026年的Letta MemFS相对于早期Memory Blocks有什么结构性变化？Git能够帮助核验什么，又不能证明哪些事实？
4. MemGPT论文DMR表中GPT-4固定基线Accuracy为32.1%，GPT-4+MemGPT为92.5%。差值是多少百分点？能否把差异完全归因成“自主管理模块”本身？为什么？
5. 课堂任务单轮直接输入4000/输出500；两次查询各输入800/输出120；最后回答输入4800/输出500。计算总输入、总输出和相对单轮额外消耗。为什么还要测延迟与质量？
6. 第30天问“P项目能否收费”，D04有AI建议，D12有暂停，D26有Q的一次许可。请写出从记忆检索到控制器动作的最小可靠步骤，并指出MemGPT论文架构哪部分不是直接的授权核验器。

第十一部分｜逐题解析①

思考题详解（1—3）

1｜所谓“虚拟上下文”

外部数据库可保存远多于单轮Token上限的历史；模型每次真正读取的仍是有限Main Context。当需要旧证据时，智能体调用工具取回一部分并重新送入下一次推理，形成较大“可利用历史”的体验。比如P项目D12暂停虽然不在第30天近期对话窗口，但通过Recall检索能够重新加入本轮。

2｜系统五个主要区域

Main Context：①系统指令，告诉模型怎样使用工具与处理事件；②Working Context，可编辑的重要短文本；③FIFO Queue，近期消息、工具结果以及历史滚动摘要。External Context：④Recall Storage，原始交互消息；⑤Archival Storage，归档知识/文档片段。只有前三者直接构成本轮输入，后两者须经工具取回才会被读取。

3｜不同信息的存储选择

D12有效暂停：若需经常判断，可在Working Context放简短状态及来源指针，但权威记录与原始事件仍应持久化；D09的完整HTTP日志：存Recall或事件档案更合适，以便保留原字节；外部论文段落：可放Archival Storage或有索引的文件系统。选择并非互斥，关键是不要用摘要取代原始证据。

> **要点：**常驻摘要是便利的读取缓存，真正的当前授权仍应依据正式来源验证。

第十一部分｜逐题解析②

思考题详解（4—6）

4｜内存压力处理

在容量预警附近，队列管理器提醒模型还有机会保存重要历史；达到刷新阈值时，淘汰部分旧FIFO并用这些消息和旧摘要生成递归新摘要。旧消息不再直接处于上下文，但论文设计让它们保存在Recall Storage。新摘要不能保证逐句保留；有丢失时要在外部历史中重新搜索。

5｜16,000 Token课堂计算

旧占用2,800+700+9,600+1,400=14,500；增加2,000后成为16,500，超过上限500。新FIFO=9,600−6,000+2,000=5,600，刷新后总量2,800+400+5,600+1,400=10,200，剩余16,000−10,200=5,800 Token。可见节省了空间，但语义完整性要单独检查。

6｜Function Executor职责

LLM产生命名工具调用及参数，Function Executor校验并实际访问外部系统，再返回成功/失败或检索结果。模型“打算检索”既不证明已查询成功，也不证明结果真实可靠；工具执行完成同样不必然等于业务目标达成。关键项目行动还需权威许可和环境状态核验。

> **要点：**容量计算是教学示意；论文给出的警告、刷新与淘汰比例是可配置思想的例子。

第十一部分｜逐题解析③

思考题详解（7—9）

7｜Heartbeat为什么存在

它控制函数返回之后是否立即再让LLM运行，用于多跳取证。示例：第一次检索D12暂停的原始决定，第二次检索D26是否对P有例外。发现Q的许可与P无关后停止；如关键当前状态未能证实则返回“不足以确认”，而不是无限继续调用。

8｜Memory Blocks是什么

它是可持久保存到Agent状态的命名文本块，在使用时通常重新放入模型提示词，所以占用Token；它既不是底座模型的参数权重，也不等于完整聊天历史。一个Block可记录常用有效偏好，消息库则保留详细的时序交互。

9｜MemFS与Git

MemFS将记忆组织成可读写的文件及目录，Git记录修改、支持差异和分支合并；这让修改较可审计。但Git提交不能独立保证主张真实、来源正确、语义不冲突，也不能提供真实费用授权。2026年这些能力来自Letta产品新演进，不是2023年论文原有形式。

> **要点：**区分“内容是否被保存”和“内容是否正确”是这三题最重要的共同原则。

第十一部分｜逐题解析④

思考题详解（10—12）

10｜原论文指标的正确解释

92.5−32.1=60.4个百分点（不是60.4%的相对增长）。它对应作者DMR数据与对照协议：MemGPT可回访更详细的历史，对照看到有损摘要，并可能采用不同数量的检索与模型调用。因此不能仅把全部差异归结为“自编辑控制器”单模块贡献。

11｜生命周期Token增量

两次查询分别消耗800输入、120输出，之后最终回答4,800输入、500输出，所以总输入800×2+4,800=6,400，总输出120×2+500=740；对比一次回答4,000/500，增加2,400输入和240输出。模型响应延迟、搜索I/O、任务正确率、失败重试及写入维护费用还需要另行统计。

12｜P项目的最小可靠决策过程

先识别项目P与当前时间；查P的权威现行状态及D12原始决定；检查D26许可是否属于P（实际上只对Q有效）；在证据不足时保留未知；如果只是回答则给出“目前未见P新许可”的证据与限定；若要实际执行收费调用则必须经控制器独立验证费用和授权。原论文Memory Function Executor负责函数执行与状态管理，但不等于专为科研预算设计的审批系统。

> **要点：**完成本题应能够独立画出“查历史→核状态→解释→控制→真实执行”的完整链，不把AI建议当批准。

第十二部分 | 引用与进一步阅读

参考文献：原论文、Letta历史与经典记忆接口

本讲将公开原始论文、官方产品说明与课程推演明确分开。以下[R#]标记用于讲义内回核原文，读到准确实验数据或具体软件功能时，应以相应年份与版本的源资料为准。

[R1] Packer, C. et al. (2023, revised 2024). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. 原始架构、Queue Manager、控制流、DMR与Nested KV。
https://ar5iv.labs.arxiv.org/html/2310.08560

[R2] Letta (2024-09-23). MemGPT Is Now Part of Letta. 官方名称演变与定位。
https://www.letta.com/blog/memgpt-and-letta/

[R3] Letta Documentation. Memory Blocks (Core Memory). 经典/SDK记忆块、持久化与上下文注入。
https://docs.letta.com/v1-sdk/memory/memory-blocks

[R4] Letta (2025-05-14). Memory Blocks: The Key to Agentic Context Management. Block label/value/limit与可编辑性。
https://www.letta.com/blog/memory-blocks/

[R5] Letta Documentation. Agents API Reference. API的block、compaction、agent state等产品定义。
https://docs.letta.com/api/resources/agents

> **论文证据与推断：**官方文档会更新；本讲关于2026年产品的描述按查阅时点2026-10-19记录，不应反推到早期论文设置。

第十二部分｜版本与开放问题

参考文献：2026年MemFS、背景研究与学习结束

[R6] Letta (2026-02-12). Introducing Context Repositories: Git-based Memory for Coding Agents. 官方产品研究博客。https://www.letta.com/blog/context-repositories/

[R7] Letta Documentation. MemFS — Git-backed Memory Filesystem. 目录/文件/版本概念。https://docs.letta.com/concepts/memfs

[R8] Letta Documentation. Memory & Dreaming. 后台记忆整理与产品接口。https://docs.letta.com/configuration/memory

[R9] Wooders, S. / Letta (2026-04-03). Why Memory Isn’t a Plugin. Stateful harness 与只检索的区别（工业观点）。https://www.letta.com/blog/why-memory-isnt-a-plugin/

[R10] Liu, N. F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL. https://arxiv.org/abs/2307.03172

[R11] Wu, D. et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. https://arxiv.org/abs/2410.10813

[R12] MemGPT original research homepage (paper, code, data entry). https://research.memgpt.ai/

最后，请独立重建这套系统

- 画出MemGPT的系统指令、Working Context、FIFO/摘要、Recall、Archival和工具执行器。
- 用你自己的话说明：为什么外存很大，不代表模型本轮可见信息无限?
- 说明Letta Memory Blocks与MemFS分别带来什么工程能力，不能保证什么。
- 用一个反例证明“检索正确”与“实际业务正确执行”之间仍存在断层。

> **要点：**下一讲将研究 Mem0、A-MEM 等更偏向动态记忆组织的方法。它们与MemGPT强调的“上下文调度”目标有所不同。